Product Extractor Generator

Autocoder User Guide

1 Description

The purpose of this generator is to provide a user the ability to extract data from packet and create data products out of them. The generator takes a YAML model file as an input which is used by the component to determine which packets to extract data products from as well as the offset and type in the packet the component needs to extract. This is performed through an Ada specification and implementation that is autocoded to perform the extraction and verification of the data product. In the case that extracting a data product is invalid for the specified type, then an invalid data structure is returned with the value of the invalid product.

Note the example shown in this documentation is used in the unit test of this component so that the reader of this document can see it being used in context. Please refer to the unit test code for more details on how this generator can be used.

2 Schema

The following pykwalify schema is used to validate the input YAML model. The schema is commented to show what each of the available YAML keys are and what they accomplish. Even without knowing the specifics of pykwalify schemas, you should be able to gleam some knowledge from the file below.

```
---
# This schema describes the yaml format for a product extractor list.
type: map
mapping:
  # Description of the data products from all packets to be extracted.
  description:
    type: str
    required: False
  # List of data products to include in the suite.
  data_products:
    seq:
      - type: map
        mapping:
          # Name of the product.
          name:
            type: str
            required: True
          # Description of the data_products item.
          description:
            type: str
            required: False
          # Type of the product.
          type:
            type: str
            required: True
          # Apid of the product in the associated packet.
```

```
          apid:
            type: int
            required: True
          # Offset of the product in the associated packet.
          offset:
            type: int
            required: True
          # Time format of the packet.
          time:
            type: str
            enum: ['current_time', 'packet_time']
            required: True
          # Optional default value for the product, supplied as an Ada aggregate
          # string for the product's packed type (e.g. "(Value => 0)"). When
          # present, the product extractor seeds this value into the data
          ↪  product
          # database during Set_Up, guaranteeing the product exists before an
          # assembly fully starts executing. The string is validated against the
          # product type by the Ada compiler. When omitted, no default is
          ↪  seeded.
          default_at_set_up:
            type: str
            required: False
    # A data product extractor must have at least one product to extract.
    range:
      min: 1
    required: True
```

3 Example Input

The following is an example extracted product input yaml file. Model files must be named in the form *assembly_name.extracted_products.yaml* where the specific name of the extracted products is not allowed. The *assembly_name* is the assembly which this table will be used, and the rest of the model file name must remain as shown. Generally this file is created in the same directory or near to the assembly model file. This example adheres to the schema shown in the previous section, and is commented to give clarification.

```
---
data_products:
  - name: Test_Product_1
    type: Packed_U16.T
    apid: 100
    offset: 10
    time: packet_time
  - name: Test_Product_2
    type: Packed_Byte.T
    apid: 100
    offset: 16
    time: current_time
  - name: Test_Product_3
    type: Packed_U32.T
    apid: 200
    offset: 8
    time: current_time
    default_at_set_up: "(Value => 42)"
  - name: Test_Product_4
    type: Packed_Natural.T
    apid: 200
```

```
    offset: 20
    time: packet_time
  - name: Test_Product_5
    description: Test product for the little endian version of a packed type.
    type: Packed_Natural.T_Le
    apid: 300
    offset: 15
    time: packet_time
    default_at_set_up: "(Value => 7)"
```

The specified data products consist of everything that the user would normally be required to include when defining a data product for a component, with the addition of the offset in the packet and corresponding APID. The type is also required which must be in the form of a packed type. The last required field is the time type which is either the current time of extraction from the packet or the time contained in the packet.

4 Seeding Default Values at Set Up

Because the component only produces a data product when a matching packet is received, a product is not published until its first packet arrives. In a common setup the component's data products are stored in a data product database and read by other components as data dependencies; such a consumer would receive a `Not_Available` status at start up for any product that has not yet been published. To close this gap, a product may optionally specify a `default_at_set_up` value in its model entry. This is an Ada aggregate string for the product's packed type (for example `"(Value => 0)"` for a `Packed_U32.T`). The string is dropped verbatim into the generated code and is therefore validated against the product type by the Ada compiler.

When a product declares a `default_at_set_up`, the generator emits a builder function for it and attaches that builder to the product's entry in the extraction list. During the component's `Set_Up` procedure the component publishes one data product per declared default, stamped with the current system time, so a value is available for every seeded product before the first packet is processed. Products that do not declare a default are not seeded.

5 Example Output

The example input shown in the previous section produces the following Ada output. The `Data_Product_Extraction_List` variable should be passed into the CCSDS Product Extractor component's `Init` procedure during assembly initialization.

The main job of the generator here was to verify the input YAML for validity and then to translate the data to an Ada data structure and Ada extraction and verification function for each product for use by the component. The generator also dynamically creates the data products for the component based on the YAML input.

Ads file:

```
-- Standard includes:
with Product_Extractor_Types; use Product_Extractor_Types;
with Data_Product;
with Data_Product_Types; use Data_Product_Types;
with Ccsds_Space_Packet;
with Invalid_Product_Data;
with Sys_Time;

package Test_Products is

   function Extract_And_Validate_Test_Product_1 (Pkt : in Ccsds_Space_Packet.T;
   ↪  Id_Base : in Data_Product_Types.Data_Product_Id; Timestamp : in
   ↪  Sys_Time.T; Dp : out Data_Product.T; Invalid_Data_Product : out
   ↪  Invalid_Product_Data.T) return Product_Status;
```

```
12
13    function Extract_And_Validate_Test_Product_2 (Pkt : in Ccsds_Space_Packet.T;
      ↪  Id_Base : in Data_Product_Types.Data_Product_Id; Timestamp : in
      ↪  Sys_Time.T; Dp : out Data_Product.T; Invalid_Data_Product : out
      ↪  Invalid_Product_Data.T) return Product_Status;
14
15    function Extract_And_Validate_Test_Product_3 (Pkt : in Ccsds_Space_Packet.T;
      ↪  Id_Base : in Data_Product_Types.Data_Product_Id; Timestamp : in
      ↪  Sys_Time.T; Dp : out Data_Product.T; Invalid_Data_Product : out
      ↪  Invalid_Product_Data.T) return Product_Status;
16
17    function Extract_And_Validate_Test_Product_4 (Pkt : in Ccsds_Space_Packet.T;
      ↪  Id_Base : in Data_Product_Types.Data_Product_Id; Timestamp : in
      ↪  Sys_Time.T; Dp : out Data_Product.T; Invalid_Data_Product : out
      ↪  Invalid_Product_Data.T) return Product_Status;
18
19    -- Test product for the little endian version of a packed type.
20    function Extract_And_Validate_Test_Product_5 (Pkt : in Ccsds_Space_Packet.T;
      ↪  Id_Base : in Data_Product_Types.Data_Product_Id; Timestamp : in
      ↪  Sys_Time.T; Dp : out Data_Product.T; Invalid_Data_Product : out
      ↪  Invalid_Product_Data.T) return Product_Status;
21
22    -- Builders for the default ("seed") values of products that requested a
23    -- "default_at_set_up" in the model. The component publishes these during
24    -- Set_Up so the products exist in the data product database before the first
25    -- packet is processed.
26    function Make_Default_Test_Product_3 (Id_Base : in
      ↪  Data_Product_Types.Data_Product_Id; Timestamp : in Sys_Time.T) return
      ↪  Data_Product.T;
27    function Make_Default_Test_Product_5 (Id_Base : in
      ↪  Data_Product_Types.Data_Product_Id; Timestamp : in Sys_Time.T) return
      ↪  Data_Product.T;
28
29    -- The list of products to extract from each apid, paired with their optional
      ↪  set-up default builder:
30    Extract_Products_100 : aliased Extractor_List := [
31       0 => (Extract => Extract_And_Validate_Test_Product_1'Access, Make_Default
         ↪  => null),
32       1 => (Extract => Extract_And_Validate_Test_Product_2'Access, Make_Default
         ↪  => null)
33    ];
34    Extract_Products_200 : aliased Extractor_List := [
35       0 => (Extract => Extract_And_Validate_Test_Product_3'Access, Make_Default
         ↪  => Make_Default_Test_Product_3'Access),
36       1 => (Extract => Extract_And_Validate_Test_Product_4'Access, Make_Default
         ↪  => null)
37    ];
38    Extract_Products_300 : aliased Extractor_List := [
39       0 => (Extract => Extract_And_Validate_Test_Product_5'Access, Make_Default
         ↪  => Make_Default_Test_Product_5'Access)
40    ];
41
42    -- Initial product extraction list containing the information to extract all
      ↪  the products requested by each apid
43    Data_Product_Extraction_List : aliased Extracted_Product_List := [
44       0 => (
45          Apid => 100,
46          Extract_List => Extract_Products_100'Access
47       ),
48       1 => (
49          Apid => 200,
```

```
         Extract_List => Extract_Products_200'Access
      ),
      2 => (
         Apid => 300,
         Extract_List => Extract_Products_300'Access
      )
   ];
   Data_Product_Extraction_List_Access : constant Extracted_Product_List_Access
   ↪  := Data_Product_Extraction_List'Access;

end Test_Products;
```

Adb file:

```
-- Standard includes:
with Basic_Types;
with Byte_Array_Util;
with Ccsds_Primary_Header; use Ccsds_Primary_Header;
with Interfaces; use Interfaces;
with Extract_Data_Product; use Extract_Data_Product;
with Packed_U16;
with Packed_U16.Validation;
with Packed_Byte;
with Packed_Byte.Validation;
with Packed_U32;
with Packed_U32.Validation;
with Packed_Natural;
with Packed_Natural.Validation;

package body Test_Products is

   -- The implementation for each extract and validate of each extracted data
   ↪  product
   function Extract_And_Validate_Test_Product_1 (Pkt : in Ccsds_Space_Packet.T;
   ↪  Id_Base : in Data_Product_Types.Data_Product_Id; Timestamp : in
   ↪  Sys_Time.T; Dp : out Data_Product.T; Invalid_Data_Product : out
   ↪  Invalid_Product_Data.T) return Product_Status is
      Local_Id : constant Data_Product_Types.Data_Product_Id := 0;
      Id : constant Data_Product_Types.Data_Product_Id := Id_Base + Local_Id;
      Extraction_Status : Extract_Status;
      Ignore : Sys_Time.T renames Timestamp;
   begin
      pragma Assert (Pkt.Header.Apid = 100);

      -- Initialize out parameters:
      Invalid_Data_Product := (
         Id => Data_Product_Types.Data_Product_Id'First,
         Errant_Field_Number => 0,
         Errant_Field => [others => 0]
      );

      -- Use the generic extraction function using the autocoded values from the
      ↪  YAML file to get the information needed from the packet
      Extraction_Status := Extract_Data_Product.Extract_Data_Product (Pkt =>
      ↪  Pkt, Offset => 10, Length => Packed_U16.Size_In_Bytes, Id => Id,
      ↪  Timestamp => Sys_Time.Serialization.From_Byte_Array (Pkt.Data
      ↪  (Pkt.Data'First .. Pkt.Data'First +
      ↪  Sys_Time.Serialization.Serialized_Length - 1)), Dp => Dp);

      -- Make sure the extraction was successful, at this point the only failure
      ↪  should be the length
```

```
38          case Extraction_Status is
39             when Length_Overflow => return Length_Error;
40             when Success => null;
41          end case;
42
43          -- Now make sure the data product is valid for the type that we are
            ↪  extracting using the validation from the packed type generation.
44          -- Validate directly on the byte buffer to avoid passing
            ↪  potentially-invalid typed data.
45          declare
46             Ef : Unsigned_32;
47             Dp_Bytes : Packed_U16.Serialization.Byte_Array renames Dp.Buffer (
48                Dp.Buffer'First .. Dp.Buffer'First +
                   ↪  Packed_U16.Serialization.Byte_Array'Length - 1
49             );
50          begin
51             pragma Warnings (Off, "this code can never be executed and has been
                ↪  deleted");
52             if Packed_U16.Always_Valid or else
53                Packed_U16.Validation.Valid (Bytes => Dp_Bytes, Errant_Field => Ef)
54             then
55                return Success;
56             else
57                -- When there is a validation error, fill in a data structure with
                   ↪  the relevant information for the component to use to send an
                   ↪  event.
58                declare
59                   P_Type : Basic_Types.Poly_Type := [others => 0];
60                begin
61                   -- Copy extracted value into poly type
62                   Byte_Array_Util.Safe_Right_Copy (P_Type, Pkt.Data (10 .. 10 +
                      ↪  Packed_U16.Size_In_Bytes - 1));
63                   Invalid_Data_Product.Id := Id;
64                   Invalid_Data_Product.Errant_Field_Number := Ef;
65                   Invalid_Data_Product.Errant_Field := P_Type;
66                end;
67                return Invalid_Data;
68             end if;
69             pragma Warnings (On, "this code can never be executed and has been
                ↪  deleted");
70          end;
71       end Extract_And_Validate_Test_Product_1;
72
73       function Extract_And_Validate_Test_Product_2 (Pkt : in Ccsds_Space_Packet.T;
         ↪  Id_Base : in Data_Product_Types.Data_Product_Id; Timestamp : in
         ↪  Sys_Time.T; Dp : out Data_Product.T; Invalid_Data_Product : out
         ↪  Invalid_Product_Data.T) return Product_Status is
74          Local_Id : constant Data_Product_Types.Data_Product_Id := 1;
75          Id : constant Data_Product_Types.Data_Product_Id := Id_Base + Local_Id;
76          Extraction_Status : Extract_Status;
77       begin
78          pragma Assert (Pkt.Header.Apid = 100);
79
80          -- Initialize out parameters:
81          Invalid_Data_Product := (
82             Id => Data_Product_Types.Data_Product_Id'First,
83             Errant_Field_Number => 0,
84             Errant_Field => [others => 0]
85          );
86
87          -- Use the generic extraction function using the autocoded values from the
            ↪  YAML file to get the information needed from the packet
```

```
88       Extraction_Status := Extract_Data_Product.Extract_Data_Product (Pkt =>
         ↪  Pkt, Offset => 16, Length => Packed_Byte.Size_In_Bytes, Id => Id,
         ↪  Timestamp => Timestamp, Dp => Dp);
89
90       -- Make sure the extraction was successful, at this point the only failure
         ↪  should be the length
91       case Extraction_Status is
92          when Length_Overflow => return Length_Error;
93          when Success => null;
94       end case;
95
96       -- Now make sure the data product is valid for the type that we are
         ↪  extracting using the validation from the packed type generation.
97       -- Validate directly on the byte buffer to avoid passing
         ↪  potentially-invalid typed data.
98       declare
99          Ef : Unsigned_32;
100         Dp_Bytes : Packed_Byte.Serialization.Byte_Array renames Dp.Buffer (
101            Dp.Buffer'First .. Dp.Buffer'First +
               ↪  Packed_Byte.Serialization.Byte_Array'Length - 1
102         );
103      begin
104         pragma Warnings (Off, "this code can never be executed and has been
            ↪  deleted");
105         if Packed_Byte.Always_Valid or else
106            Packed_Byte.Validation.Valid (Bytes => Dp_Bytes, Errant_Field => Ef)
107         then
108            return Success;
109         else
110            -- When there is a validation error, fill in a data structure with
               ↪  the relevant information for the component to use to send an
               ↪  event.
111            declare
112               P_Type : Basic_Types.Poly_Type := [others => 0];
113            begin
114               -- Copy extracted value into poly type
115               Byte_Array_Util.Safe_Right_Copy (P_Type, Pkt.Data (16 .. 16 +
                  ↪  Packed_Byte.Size_In_Bytes - 1));
116               Invalid_Data_Product.Id := Id;
117               Invalid_Data_Product.Errant_Field_Number := Ef;
118               Invalid_Data_Product.Errant_Field := P_Type;
119            end;
120            return Invalid_Data;
121         end if;
122         pragma Warnings (On, "this code can never be executed and has been
            ↪  deleted");
123      end;
124   end Extract_And_Validate_Test_Product_2;
125
126   function Extract_And_Validate_Test_Product_3 (Pkt : in Ccsds_Space_Packet.T;
      ↪  Id_Base : in Data_Product_Types.Data_Product_Id; Timestamp : in
      ↪  Sys_Time.T; Dp : out Data_Product.T; Invalid_Data_Product : out
      ↪  Invalid_Product_Data.T) return Product_Status is
127      Local_Id : constant Data_Product_Types.Data_Product_Id := 2;
128      Id : constant Data_Product_Types.Data_Product_Id := Id_Base + Local_Id;
129      Extraction_Status : Extract_Status;
130   begin
131      pragma Assert (Pkt.Header.Apid = 200);
132
133      -- Initialize out parameters:
134      Invalid_Data_Product := (
```

```
135            Id => Data_Product_Types.Data_Product_Id'First,
136            Errant_Field_Number => 0,
137            Errant_Field => [others => 0]
138         );
139
140         -- Use the generic extraction function using the autocoded values from the
            ↪  YAML file to get the information needed from the packet
141         Extraction_Status := Extract_Data_Product.Extract_Data_Product (Pkt =>
            ↪  Pkt, Offset => 8, Length => Packed_U32.Size_In_Bytes, Id => Id,
            ↪  Timestamp => Timestamp, Dp => Dp);
142
143         -- Make sure the extraction was successful, at this point the only failure
            ↪  should be the length
144         case Extraction_Status is
145            when Length_Overflow => return Length_Error;
146            when Success => null;
147         end case;
148
149         -- Now make sure the data product is valid for the type that we are
            ↪  extracting using the validation from the packed type generation.
150         -- Validate directly on the byte buffer to avoid passing
            ↪  potentially-invalid typed data.
151         declare
152            Ef : Unsigned_32;
153            Dp_Bytes : Packed_U32.Serialization.Byte_Array renames Dp.Buffer (
154               Dp.Buffer'First .. Dp.Buffer'First +
                  ↪  Packed_U32.Serialization.Byte_Array'Length - 1
155            );
156         begin
157            pragma Warnings (Off, "this code can never be executed and has been
               ↪  deleted");
158            if Packed_U32.Always_Valid or else
159               Packed_U32.Validation.Valid (Bytes => Dp_Bytes, Errant_Field => Ef)
160            then
161               return Success;
162            else
163               -- When there is a validation error, fill in a data structure with
                  ↪  the relevant information for the component to use to send an
                  ↪  event.
164               declare
165                  P_Type : Basic_Types.Poly_Type := [others => 0];
166               begin
167                  -- Copy extracted value into poly type
168                  Byte_Array_Util.Safe_Right_Copy (P_Type, Pkt.Data (8 .. 8 +
                     ↪  Packed_U32.Size_In_Bytes - 1));
169                  Invalid_Data_Product.Id := Id;
170                  Invalid_Data_Product.Errant_Field_Number := Ef;
171                  Invalid_Data_Product.Errant_Field := P_Type;
172               end;
173               return Invalid_Data;
174            end if;
175            pragma Warnings (On, "this code can never be executed and has been
               ↪  deleted");
176         end;
177      end Extract_And_Validate_Test_Product_3;
178
179      function Extract_And_Validate_Test_Product_4 (Pkt : in Ccsds_Space_Packet.T;
         ↪  Id_Base : in Data_Product_Types.Data_Product_Id; Timestamp : in
         ↪  Sys_Time.T; Dp : out Data_Product.T; Invalid_Data_Product : out
         ↪  Invalid_Product_Data.T) return Product_Status is
180         Local_Id : constant Data_Product_Types.Data_Product_Id := 3;
```

```
      Id : constant Data_Product_Types.Data_Product_Id := Id_Base + Local_Id;
      Extraction_Status : Extract_Status;
      Ignore : Sys_Time.T renames Timestamp;
   begin
      pragma Assert (Pkt.Header.Apid = 200);

      -- Initialize out parameters:
      Invalid_Data_Product := (
         Id => Data_Product_Types.Data_Product_Id'First,
         Errant_Field_Number => 0,
         Errant_Field => [others => 0]
      );

      -- Use the generic extraction function using the autocoded values from the YAML file to get the information needed from the packet
      Extraction_Status := Extract_Data_Product.Extract_Data_Product (Pkt => Pkt, Offset => 20, Length => Packed_Natural.Size_In_Bytes, Id => Id, Timestamp => Sys_Time.Serialization.From_Byte_Array (Pkt.Data (Pkt.Data'First .. Pkt.Data'First + Sys_Time.Serialization.Serialized_Length - 1)), Dp => Dp);

      -- Make sure the extraction was successful, at this point the only failure should be the length
      case Extraction_Status is
         when Length_Overflow => return Length_Error;
         when Success => null;
      end case;

      -- Now make sure the data product is valid for the type that we are extracting using the validation from the packed type generation.
      -- Validate directly on the byte buffer to avoid passing potentially-invalid typed data.
      declare
         Ef : Unsigned_32;
         Dp_Bytes : Packed_Natural.Serialization.Byte_Array renames Dp.Buffer (
            Dp.Buffer'First .. Dp.Buffer'First + Packed_Natural.Serialization.Byte_Array'Length - 1
         );
      begin
         pragma Warnings (Off, "this code can never be executed and has been deleted");
         if Packed_Natural.Always_Valid or else
            Packed_Natural.Validation.Valid (Bytes => Dp_Bytes, Errant_Field => Ef)
         then
            return Success;
         else
            -- When there is a validation error, fill in a data structure with the relevant information for the component to use to send an event.
            declare
               P_Type : Basic_Types.Poly_Type := [others => 0];
            begin
               -- Copy extracted value into poly type
               Byte_Array_Util.Safe_Right_Copy (P_Type, Pkt.Data (20 .. 20 + Packed_Natural.Size_In_Bytes - 1));
               Invalid_Data_Product.Id := Id;
               Invalid_Data_Product.Errant_Field_Number := Ef;
               Invalid_Data_Product.Errant_Field := P_Type;
            end;
            return Invalid_Data;
```

```
228            end if;
229            pragma Warnings (On, "this code can never be executed and has been
               ↪  deleted");
230         end;
231      end Extract_And_Validate_Test_Product_4;
232
233      function Extract_And_Validate_Test_Product_5 (Pkt : in Ccsds_Space_Packet.T;
         ↪  Id_Base : in Data_Product_Types.Data_Product_Id; Timestamp : in
         ↪  Sys_Time.T; Dp : out Data_Product.T; Invalid_Data_Product : out
         ↪  Invalid_Product_Data.T) return Product_Status is
234         Local_Id : constant Data_Product_Types.Data_Product_Id := 4;
235         Id : constant Data_Product_Types.Data_Product_Id := Id_Base + Local_Id;
236         Extraction_Status : Extract_Status;
237         Ignore : Sys_Time.T renames Timestamp;
238      begin
239         pragma Assert (Pkt.Header.Apid = 300);
240
241         -- Initialize out parameters:
242         Invalid_Data_Product := (
243            Id => Data_Product_Types.Data_Product_Id'First,
244            Errant_Field_Number => 0,
245            Errant_Field => [others => 0]
246         );
247
248         -- Use the generic extraction function using the autocoded values from the
            ↪  YAML file to get the information needed from the packet
249         Extraction_Status := Extract_Data_Product.Extract_Data_Product (Pkt =>
            ↪  Pkt, Offset => 15, Length => Packed_Natural.Size_In_Bytes, Id => Id,
            ↪  Timestamp => Sys_Time.Serialization.From_Byte_Array (Pkt.Data
            ↪  (Pkt.Data'First .. Pkt.Data'First +
            ↪  Sys_Time.Serialization.Serialized_Length - 1)), Dp => Dp);
250
251         -- Make sure the extraction was successful, at this point the only failure
            ↪  should be the length
252         case Extraction_Status is
253            when Length_Overflow => return Length_Error;
254            when Success => null;
255         end case;
256
257         -- Now make sure the data product is valid for the type that we are
            ↪  extracting using the validation from the packed type generation.
258         -- Validate directly on the byte buffer to avoid passing
            ↪  potentially-invalid typed data.
259         declare
260            Ef : Unsigned_32;
261            Dp_Bytes : Packed_Natural.Serialization_Le.Byte_Array renames Dp.Buffer
               ↪  (
262               Dp.Buffer'First .. Dp.Buffer'First +
                  ↪  Packed_Natural.Serialization_Le.Byte_Array'Length - 1
263            );
264         begin
265            pragma Warnings (Off, "this code can never be executed and has been
               ↪  deleted");
266            if Packed_Natural.Always_Valid or else
267               Packed_Natural.Validation.Valid_Le (Bytes => Dp_Bytes, Errant_Field
                  ↪  => Ef)
268            then
269               return Success;
270            else
271               -- When there is a validation error, fill in a data structure with
                  ↪  the relevant information for the component to use to send an
                  ↪  event.
```

```
272             declare
273                P_Type : Basic_Types.Poly_Type := [others => 0];
274             begin
275                -- Copy extracted value into poly type
276                Byte_Array_Util.Safe_Right_Copy (P_Type, Pkt.Data (15 .. 15 +
                    ↪  Packed_Natural.Size_In_Bytes - 1));
277                Invalid_Data_Product.Id := Id;
278                Invalid_Data_Product.Errant_Field_Number := Ef;
279                Invalid_Data_Product.Errant_Field := P_Type;
280             end;
281             return Invalid_Data;
282          end if;
283          pragma Warnings (On, "this code can never be executed and has been
               ↪  deleted");
284       end;
285    end Extract_And_Validate_Test_Product_5;
286
287    -- Build the default ("seed") value for Test_Product_3. The default
288    -- value string from the model is validated against the product type by the
289    -- compiler.
290    function Make_Default_Test_Product_3 (Id_Base : in
       ↪  Data_Product_Types.Data_Product_Id; Timestamp : in Sys_Time.T) return
       ↪  Data_Product.T is
291       Local_Id : constant Data_Product_Types.Data_Product_Id := 2;
292       Id : constant Data_Product_Types.Data_Product_Id := Id_Base + Local_Id;
293       Dp : Data_Product.T := (
294          Header => (Time => Timestamp, Id => Id, Buffer_Length =>
               ↪  Packed_U32.Serialization.Serialized_Length),
295          Buffer => [others => 0]
296       );
297    begin
298       Dp.Buffer (Dp.Buffer'First .. Dp.Buffer'First +
           ↪  Packed_U32.Serialization.Serialized_Length - 1) :=
299          Packed_U32.Serialization.To_Byte_Array ((Value => 42));
300       return Dp;
301    end Make_Default_Test_Product_3;
302
303    -- Build the default ("seed") value for Test_Product_5. The default
304    -- value string from the model is validated against the product type by the
305    -- compiler.
306    function Make_Default_Test_Product_5 (Id_Base : in
       ↪  Data_Product_Types.Data_Product_Id; Timestamp : in Sys_Time.T) return
       ↪  Data_Product.T is
307       Local_Id : constant Data_Product_Types.Data_Product_Id := 4;
308       Id : constant Data_Product_Types.Data_Product_Id := Id_Base + Local_Id;
309       Dp : Data_Product.T := (
310          Header => (Time => Timestamp, Id => Id, Buffer_Length =>
               ↪  Packed_Natural.Serialization_Le.Serialized_Length),
311          Buffer => [others => 0]
312       );
313    begin
314       Dp.Buffer (Dp.Buffer'First .. Dp.Buffer'First +
           ↪  Packed_Natural.Serialization_Le.Serialized_Length - 1) :=
315          Packed_Natural.Serialization_Le.To_Byte_Array ((Value => 7));
316       return Dp;
317    end Make_Default_Test_Product_5;
318
319 end Test_Products;
```